

Discord Multi-Bot + LLM Wiki + Obsidian

AI-Powered Personal Knowledge Operating System

by Ian Park | ianpark.vc | 주간실리콘밸리

Discord를 프론트엔드로, Claude Code를 백엔드 AI 엔진으로 사용하는 개인 지식 운영 시스템 구축 가이드.

Last updated: 2026-04-13

목차

- 1. 시스템 개요
- 2. 사전 준비
- 3. Vault 세팅
- 4. Discord 서버 & 봇 생성
- 5. 멀티봇 아키텍처
- 6. 채널 라우팅 & CLAUDE.md
- 7. LLM 위키
- 8. 자동화
- 9. 운영
- 10. 보안

1. 시스템 개요

이게 뭔가?

Discord 채널을 AI 워크스페이스로 만드는 시스템. 각 채널에 Claude Code가 붙어서, 채널마다 다른 역할을 수행하고 로컬 파일 시스템에 직접 접근한다.

Discord 서버 (내 채널들)
 ↳ MCP 플러그인
 Claude Code (N개 봇 인스턴스, tmux)
 ↳ 파일 읽기/쓰기
 로컬 Vault (마크다운 파일 + 위키)

핵심 아이디어

1. **채널 = 컨텍스트**: CLAUDE.md 하나로 채널별 역할 전환. 채널 이름을 보고 행동을 바꾼다.
2. **멀티봇 = 병렬 처리**: 봇 2~3개를 동시에 돌려서 서로 다른 채널 담당.
3. **파일시스템 = 영속성**: 모든 데이터가 로컬 마크다운. DB 없음.
4. **위키 = 합성 지식**: 원본 데이터가 들어오면, 위키 페이지를 증분 업데이트.

2. 사전 준비

```
# Claude Code CLI
# https://docs.anthropic.com/en/docs/claude-code
which claude

# tmux (멀티봇 세션 관리)
brew install tmux    # macOS
sudo apt install tmux # Linux

# Obsidian (선택 - vault 탐색/편집용)
# https://obsidian.md
```

필요한 계정: - **Anthropic** — Claude Code 구독 (Max 또는 Pro) - **Discord** — 봇 생성용 개발자 계정 -
<https://discord.com/developers/applications>

3. Vault 세팅

3.1 최소 구조

Vault 루트를 만들고, **채널** → **폴더** 매핑으로 구성한다. Discord 채널 하나당 `-data/` 폴더 하나.

```
VAULT=~/.my-vault
mkdir -p "$VAULT" && cd "$VAULT"

# 에이전트 허브 (필수)
mkdir -p agents

# 위키 (필수)
mkdir -p wiki

# 채널별 데이터 폴더 (자기 용도에 맞게)
mkdir -p channel-a-data
mkdir -p channel-b-data
mkdir -p channel-c-data
```

3.2 예시 레이아웃

```
my-vault/
├── agents/
│   ├── CLAUDE.md           # 마스터 라우팅 문서 (모든 봇이 읽음)
│   └── start-bots.sh       # 멀티봇 시작 스크립트
├── wiki/
│   ├── schema.md          # 위키 규칙
│   └── index.md            # 페이지 카탈로그
├── channel-a-data/         # Discord #channel-a에 매핑
├── channel-b-data/         # Discord #channel-b에 매핑
├── channel-c-data/         # Discord #channel-c에 매핑
└── .obsidian/              # Obsidian 자동 생성
```

이게 전부다. 채널이 늘면 폴더를 추가하면 된다. 이름 규칙은 자유 — 채널명과 폴더명만 일관되게 맞추면 됨.

3.3 원칙

원칙	이유
채널명 = 폴더명	#research → research-data/
원본 데이터는 읽기 전용	봇이 원본을 수정하면 안 됨
위키가 합성 레이어	원본 → 위키 페이지 (증분 업데이트만)

4. Discord 서버 & 봇 생성

4.1 서버 & 채널 만들기

Discord에서 서버를 만들고, 자기 업무 도메인에 맞는 채널을 추가한다. 예시:

#general	- 기본 / 자유 대화
#research	- 리서치 & 분석
#writing	- 장기 글쓰기
#ops	- 비즈니스 운영

이름은 자유. 나중에 얼마든지 추가 가능.

4.2 봇 만들기

동시 처리할 세션 수 = 봇 수. 대부분 2~3개면 충분하다.

Discord Developer Portal → <https://discord.com/developers/applications>

각 봇마다:

1. **New Application** → 이름 설정 (예: Bot-A , Bot-B)

2. **Bot 탭**:

- Reset Token → 토큰을 안전한 곳에 저장
- Privileged Gateway Intents → 3개 전부 ON:
 - Presence Intent ☒
 - Server Members Intent ☒
 - Message Content Intent ☒

3. **OAuth2 → URL Generator**:

- Scopes: bot
- Permissions: Send Messages , Read Message History , View Channels , Add Reactions
- 생성된 URL 열기 → 서버에 초대

4.3 채널 ID 확인

1. Discord 설정 → 고급 → **Developer Mode ON**
2. 채널 오른쪽클릭 → "Copy Channel ID"
3. 내 프로필 오른쪽클릭 → "Copy User ID"

메모해 둘 것 — 봇 설정에 필요하다.

5. 멀티봇 아키텍처

5.1 원리

Claude Code의 Discord 플러그인은 `DISCORD_STATE_DIR` 환경변수로 봇 토큰과 접근 제어를 격리한다. 봇마다 다른 디렉토리를 지정하면 **한 머신에서 여러 봇을 동시에 실행**할 수 있다.

```
Bot A → ~/.claude/channels/discord/      (기본값)
Bot B → ~/.claude/channels/discord-b/    (커스텀)
Bot C → ~/.claude/channels/discord-c/    (커스텀)
```

각 디렉토리에 들어가는 것:

```
~/.claude/channels/discord-b/
├── .env                # DISCORD_BOT_TOKEN=xxx
├── access.json         # 채널/유저 권한
└── inbox/              # 다운로드된 첨부파일
```

5.2 토큰 등록

각 봇의 토큰을 별도 Claude Code 세션에서 등록한다:

```
# Bot A (기본 state dir)
cd ~/my-vault/agents
claude
/discord:configure <bot-a-토큰>

# Bot B (커스텀 state dir)
DISCORD_STATE_DIR=~/.claude/channels/discord-b claude
/discord:configure <bot-b-토큰>
```

5.3 페어링

Discord에서 각 봇에게 DM을 보내면 6자리 코드가 온다.

```
# 해당 봇의 Claude Code 세션에서:
/discord:access pair <6자리코드>
```

5.4 채널 등록

각 봇에게 담당 채널을 배정한다:

```
# Bot A - #general, #research 담당
/discord:access group add <general-채널-id> --no-mention
/discord:access group add <research-채널-id> --no-mention

# Bot B - #writing, #ops 담당
/discord:access group add <writing-채널-id> --no-mention
/discord:access group add <ops-채널-id> --no-mention
```

`--no-mention` = @멘션 없이 모든 메시지에 응답. 생략하면 @봇이름 붙여야 반응.

5.5 access.json 결과

```
{
  "dmPolicy": "pairing",
  "allowFrom": ["<내-User-ID>"],
  "groups": {
    "<channel-id-1>": { "requireMention": false },
    "<channel-id-2>": { "requireMention": false }
  }
}
```

5.6 시작 스크립트

agents/start-bots.sh :

```
#!/bin/bash
set -e
WORKDIR=~/.my-vault/agents

which claude || { echo "Claude Code 미설치"; exit 1; }
which tmux || { echo "tmux 미설치"; exit 1; }

# 기존 세션 정리
tmux kill-session -t bot-a 2>/dev/null || true
tmux kill-session -t bot-b 2>/dev/null || true

# Bot A (기본 state dir)
tmux new -s bot-a -d -c "$WORKDIR" \
  'claude --dangerously-skip-permissions \
    --channels plugin:discord@claude-plugins-official'

# Bot B (커스텀 state dir)
tmux new -s bot-b -d -c "$WORKDIR" \
  'DISCORD_STATE_DIR=~/.claude/channels/discord-b \
    claude --dangerously-skip-permissions \
    --channels plugin:discord@claude-plugins-official'

echo "모든 봇 시작 완료."
tmux ls
```

핵심 플래그:

플래그	역할
<code>--dangerously-skip-permissions</code>	모든 파일/명령 자동 승인 (신뢰 환경 전용)
<code>--channels plugin:discord@claude-plugins-official</code>	Discord MCP 플러그인 활성화
<code>-c \$WORKDIR</code>	작업 디렉토리 — CLAUDE.md가 여기 있어야 함

5.7 글로벌 설정

`~/.claude/settings.json` :

```
{
  "enabledPlugins": {
    "discord@claude-plugins-official": true
  }
}
```

6. 채널 라우팅 & CLAUDE.md

6.1 CLAUDE.md의 역할

`agents/CLAUDE.md` 는 **마스터 지시서**. 모든 봇이 같은 디렉토리에서 실행되니까 같은 파일을 읽는다. 메시지가 들어오면 채널 이름을 보고 역할을 전환한다.

6.2 라우팅 테이블

CLAUDE.md에 이런 테이블을 넣는다:

```
## Channel Routing

| Channel | Role | Data Folder |
|-----|-----|-----|
| #research | Research analyst | research-data/ |
| #writing | Editor & writing partner | writing-data/ |
| #ops | Operations manager | ops-data/ |
| #general / DM | General assistant | all |

채널을 식별할 수 없으면 기본 모드로 응답.
```

6.3 채널별 지시

테이블 아래에 각 채널의 구체적 규칙을 작성한다:

#research - 리서치

데이터

- `~/my-vault/research-data/` - 저장된 기사, 노트

규칙

- 새 기사를 요약하고 위키 업데이트
- 소스 항상 인용

6.4 라이브 진행 업데이트 (선택)

긴 작업 시 Discord에 실시간으로 진행 상황을 미러링하는 패턴:

Discord 라이브 업데이트

1. ****시작****: reply로 목표 전송 → message_id 저장
2. ****진행****: edit_message로 같은 메시지에 누적 (지우지 말고 쌓기)
3. ****완료****: 새 reply로 최종 결과 (폰 알림 트리거)

간단한 답변(tool call 1~2개)은 reply 한 개로 충분.

edit_message 는 푸시 알림이 안 올린다 — 새 reply 만 알림을 트리거. 노이즈 없이 진행 상황을 보여줄 수 있다.

7. LLM 위키

7.1 개념

Karpathy 스타일 LLM 위키. 채널 폴더에서 원본 데이터가 들어오면, 봇이 읽고 지식을 추출해서 위키 페이지를 증분 업데이트한다.

Raw Sources (읽기 전용)	→	Wiki Pages (봇이 관리)	→	Output
research-data/		topics/ai.md		Discord 답변
writing-data/		people/john-doe.md		초안, 보고서

7.2 구조

```
wiki/
├─ schema.md      # 모든 봇이 따를 규칙
├─ index.md        # 전체 페이지 카탈로그 (항상 최신 유지)
├─ log.md          # 작업 기록
├─ topics/         # 주제별 페이지
└─ people/        # 인물 프로필
```

카테고리는 자유. 1~2개로 시작해서 필요할 때 추가.

7.3 핵심 규칙 (schema.md에 넣을 것)

```
# Wiki Schema

## 규칙
- 증분 업데이트만 (덮어쓰기 금지)
- 모순 발견 시 표시만 - 삭제 금지
- 페이지 추가/수정 후 index.md 갱신
- 모든 변경을 log.md에 기록
- 기밀 데이터 위키에 올리기 금지

## 페이지 포맷
# [제목]
**Last updated:** YYYY-MM-DD
**Sources:** [목록]

## Summary
## Key Facts
## Connections
```

7.4 Ingest 워크플로우

1. 원본 읽기 (예: research-data/2026-04-13-article.md)
2. index.md에서 관련 위키 페이지 찾기
3. 있으면 → 새 정보 추가
없으면 → 새 페이지 생성
4. index.md 업데이트
5. log.md에 기록

8. 자동화

8.1 모닝 브리핑 (선택)

Discord Webhook으로 매일 채널별 맞춤 브리핑을 자동 발송할 수 있다.

세팅: 1. 채널 설정 → 연동 → Webhook → 생성 → URL 복사 2. agents/briefing-config 에 저장:

```
WEBHOOK_RESEARCH=https://discord.com/api/webhooks/xxx/yyy
WEBHOOK_WRITING=https://discord.com/api/webhooks/xxx/yyy
```

3. 각 *-data/ 폴더의 최근 파일을 스캔해서 요약물 보내는 스크립트 작성.

4. cron으로 스케줄:

```
# crontab -e
0 8 * * * cd ~/my-vault/agents && python3 morning-briefing.py
```

9. 운영

명령어

```
# 전체 봇 시작
bash ~/my-vault/agents/start-bots.sh

# 상태 확인
tmux ls

# 봇 세션 접속 (로그 확인)
tmux attach -t bot-a

# 세션 분리 (봇은 계속 실행)
# Ctrl-B, D

# 봇 종료
tmux kill-session -t bot-a
```

봇 내부 명령어

```
/discord:access # 현재 설정 확인
/discord:access pair <코드> # 새 유저 페어링
/discord:access group add <id> --no-mention # 채널 추가
/discord:access group rm <id> # 채널 제거
```

메시지 흐름

```
유저가 Discord에 메시지 전송
→ Discord.js 클라이언트가 수신
→ MCP 서버가 access.json 확인
→ 권한 OK → Claude Code로 전달
→ Claude가 CLAUDE.md 읽고 채널별 역할 전환
→ 매핑된 *-data/ 폴더 접근
→ reply 도구로 응답
```

Discord 도구 목록

도구	용도
reply	메시지 전송 (파일 첨부 가능)
react	이모지 반응 추가
edit_message	봇이 보낸 메시지 수정 (진행 업데이트용)
fetch_messages	채널 히스토리 조회 (최대 100개)
download_attachment	메시지 첨부파일 다운로드

10. 보안

토큰 관리

- `.env` 파일은 자동으로 `chmod 600` 설정됨
- 절대 git에 커밋하지 말 것 — `.gitignore` 에 추가:

```
.env
**/channels/discord*/.env
briefing-config
```

`--dangerously-skip-permissions`

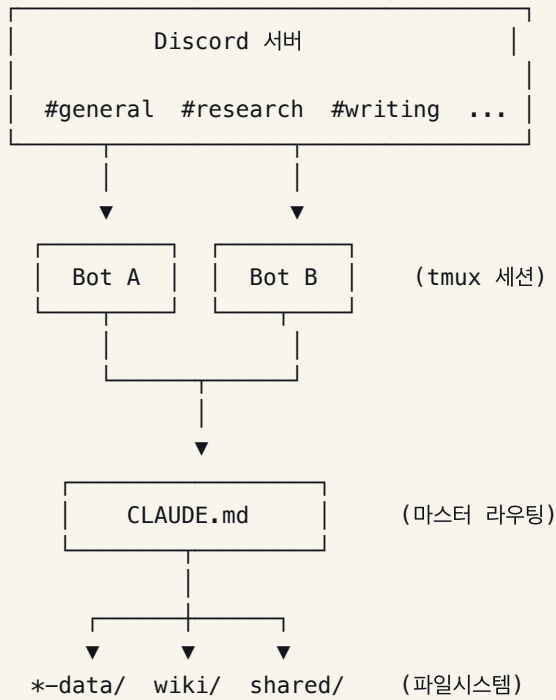
이 플래그는 모든 파일 읽기/쓰기/명령 실행을 자동 승인한다. 본인만 접근하는 신뢰 환경에서만 사용할 것.

기밀 데이터

민감한 정보를 다루는 채널이 있으면 CLAUDE.md에 명시:

```
## #ops - 운영
⚠ 기밀 - 이 채널의 데이터를 다른 곳에 노출하지 말 것.
```

아키텍처 다이어그램



멀티봇 격리

```

~/ .claude/channels/
├── discord/           ← Bot A (기본)
│   ├── .env
│   ├── access.json
│   └── inbox/
└── discord-b/        ← Bot B (커스텀 DISCORD_STATE_DIR)
    ├── .env
    ├── access.json
    └── inbox/
  
```

부록: 트러블슈팅

문제	해결
봇이 응답 안 함	<code>tmux ls</code> 로 세션 확인. <code>tmux attach</code> 로 에러 로그 확인. <code>access.json</code> 에 채널 ID 등록 여부 확인.
토큰 충돌	봇마다 별도 Discord Application + 별도 토큰 필요. 같은 토큰 = 마지막 봇만 동작.
edit_message가 너무 길어짐	Discord 2000자 제한. 새 <code>reply</code> 로 시작하고 거기에 계속 edit.
iPhone HEIC 이미지 문제	MCP 서버에 HEIC → JPEG 자동 변환 패치 적용.

부록: 확장

될	어떻게
채널 추가	Discord에서 채널 생성 → ID 복사 → <code>/discord:access group add <id></code> → CLAUDE.md 라우팅 테이블에 추가 → <code>*-data/</code> 폴더 생성
봇 추가	Discord Application 생성 → <code>DISCORD_STATE_DIR</code> 설정 → <code>start-bots.sh</code> 에 추가
위키 카테고리 추가	<code>mkdir wiki/new-category/</code> → <code>schema.md</code> 와 <code>index.md</code> 에 추가

Ian Park · [ianpark.vc](#) · 주간실리콘밸리

뉴스레터, 펀드 운영, 책 프로젝트, 사이드 벤처를 이 하나의 시스템으로 운영하면서 만든 구조를 문서화한 가이드. 자기 워크 플로우에 맞게 커스터마이징해서 쓸 것.